identity and tenancy framework that those policies would depend on. The internal platform had treated “who you are” and “which project you belong to” as properties in the same database. In a multi-tenant environment, this connection couldn’t work. Identity had to come from outside, and tenancy had to become a clear, enforceable boundary.

13.3.1 From project IDs to tenants

The team’s initial step was to upgrade the old project ID, which had informally signified customer workspaces, to an official tenant entity. Each tenant now has these three elements:

- A `tenant_id` that is globally unique `and` independent of any database key
- Its own namespace for entities, relationships, and policies
- A record in a new tenant directory that tracks metadata about ownership, configuration, and identity providers

In C³, the tenant became a first-class concept in the authorization model. Every principal, resource, and policy evaluation is connected to a tenant context. The `tenant_id,` which previously existed quietly in SQL, now appears in every authorization request, ensuring that identity, policy, and data are all consistently scoped.

13.3.2 Unified customer identity via OIDC

ACME decided early on that all customers—large or small—would authenticate using OpenID Connect (OIDC). This provides a consistent model for every tenant while keeping ACME’s internal identity system completely separate.

Two connection modes encompass all scenarios:

- Federated OIDC: enterprise customers link their own identity provider, managed through metadata stored in the tenant directory.
- Hosted OIDC: smaller tenants without an IdP use an ACME-hosted OIDC instance provisioned specifically for their tenant.